Práctica Técnica: Creación de un Componente Visual Autónomo en JavaFX (UT2 - Bloque 2)

Módulo: Desarrollo de Interfaces (2º DAM)
Modalidad: Individual
Duración estimada: 2 sesiones (3-4 horas lectivas)

---

1. Contexto y Enunciado

Hasta ahora hemos estudiado la mecánica de ensamblado con fx:root, el ciclo de vida del cargador FXML y los principios de encapsulación para evitar exponer los controles visuales internos (`Button`, `TextField`, etc.).

En esta práctica diseñaréis, construiréis e integraréis un componente visual reutilizable y autónomo desde cero, aplicando la arquitectura correcta de comunicación hacia el exterior y permitiendo su personalización mediante el estándar JavaBean para herramientas visuales como Scene Builder.

---

2. Requisitos Técnicos Obligatorios

El componente que elijáis desarrollar debe cumplir estrictamente las siguientes especificaciones:

1. **Construcción desacoplada (fx:root):**
    - El archivo `.fxml` del componente debe definir como nodo raíz `<fx:root type="..." ...>`.
    - La clase Java asociada debe heredar de dicho layout (por ejemplo, `HBox`, `VBox`, `GridPane`) y ejecutar el constructor de carga de recursos (`FXMLLoader.setRoot(this)` y `FXMLLoader.setController(this)`).
    - Empaquetar y enlazar su propia hoja de estilos (`.css`).
2. **Encapsulación y Blindaje de la Vista:**
    - Todos los controles visuales internos (`Button`, `TextField`, `Label`, etc.) deben declararse como `private`.

    - **Queda terminantemente prohibido crear getters que devuelvan referencias a nodos gráficos internos** (como `getBoton()`, `getCampoTexto()`).
3. **Lógica Interna y Reactividad:**
    - El componente debe resolver su propia operativa interna de forma aislada.
    - Debe implementar al menos un **enlace reactivo (binding) interno** (por ejemplo: deshabilitar un botón si ciertos campos están vacíos, enlazar dimensiones, etc.).
4. **Notificación de la Acción Principal:**
    - Cuando se ejecute la acción principal del componente (por ejemplo, al pulsar el botón de confirmación o interactuar con el control), el componente debe disparar un evento hacia el exterior que haga que la ventana principal capture dicho suceso e imprima un mensaje informativo en la terminal/consola.
5. **Personalización mediante Propiedades JavaBean:**
    - Debe exponer al menos **una propiedad configurable** siguiendo la tríada de métodos:
        - `xxxProperty()` (retorna el contenedor observable)
        - `getXxx()` (lectura de estado)
        - `setXxx(...)` (asignación directa e inyección declarativa)
    - Puede ser una propiedad delegada directamente a un control interno (ej. `textoBotonProperty()`) o una propiedad calculada.
6. **Aplicación de Prueba (HelloController / hello-view.fxml):**
    - Debe crearse una ventana principal que instancie el componente (vía FXML o código), configure sus propiedades visuales y verifique la salida en la terminal al activarse la acción principal.

---

3. Sugerencias de Componentes Libres

Tenéis libertad para elegir la temática del componente. Algunos ejemplos orientativos:

- **Control de Búsqueda con Filtro:** Campo de texto, ComboBox de categorías y botón de búsqueda deshabilitado si el texto está vacío. Al accionarlo, emite la acción e imprime en consola.
- **Selector Numérico / Contador (Stepper):** Botones `+` y `-` con visor central. Límite reactivo en botones. Al pulsar, notifica e imprime el valor por terminal.
- **Caja de Autenticación Rápida (PIN):** Cuatro dígitos con botón que se activa solo al completarse. Notifica la validación imprimiendo en terminal.

- **Selector de Fichero con Previsualización:** Botón para examinar y etiqueta de ruta. Notifica la acción imprimiendo en consola.

---

4. Rúbrica de Evaluación

Criterio	Excelente (4 - 5 pts)	Aceptable (2.5 - 3.9 pts)	Insuficiente (0 - 2.4 pts)
Arquitectura fx:root y FXML	Implementa correctamente `fx:root` y el constructor de carga.	Utiliza `fx:root` pero con advertencias menores en rutas o estilos.	No utiliza `fx:root` o la carga FXML está mal estructurada.
Encapsulación de la Vista	Todos los controles son `private`. No expone nodos visuales al exterior.	Controles privados, pero expone referencias o métodos redundantes.	Fuga de encapsulación grave (expone `getBoton()`, etc.).
Lógica Interna y Bindings	Resuelve su lógica interna y usa bindings reactivos coherentes.	Lógica funcional pero mediante flags manuales sin reactividad nativa.	Sin lógica interna o bindings inconsistentes/rotos.
Acción Principal y Terminal	Dispara limpiamente el evento principal y la ventana exterior imprime en consola según lo solicitado.	El evento se lanza pero con fallos de control de nulos o suscripción forzada.	No notifica al exterior ni imprime el evento en la terminal.
Estándar JavaBean	Implementa con precisión la tríada (`xxxProperty`, `getXxx`, `setXxx`) editable desde Scene Builder/FXML.	Implementa solo getter y setter clásicos o nombres discordantes.	No implementa la tríada JavaBean requerida.
Integración en Ventana	La ventana principal consume el componente a través de su API pública de forma limpia y robusta.	La integración funciona pero con acoplamientos innecesarios.	La aplicación principal no compila o falla al interactuar.

---

5. Entregables

1. **Código fuente del proyecto Maven:** Paquete `componentes` con el código Java, `.fxml` y `.css`, más la ventana principal de demostración.
2. **Breve memoria técnica (1 página):**
    - Explicación del componente propuesto y su funcionalidad.
    - Justificación de por qué la comunicación externa no vulnera la encapsulación de la vista.

6. Extra

Aplica una hoja de estilo (.css) a la aplicación y personaliza los elementos de tu componente.
Este apartado se tendrá en cuenta en la nota de participación en clase.